



CSE 4205

Digital Logic Design

Gate Level Minimization

Course Teacher: Md. Hamjajul Ashmafee

Lecturer, CSE, IUT

Email: ashmafee@iut-dhaka.edu



Remaining to Add

- Review other books and internet resources



To be noted more:

- Circuit diagrams of NAND and NOR implementation
- 3.7 – other two level implementation
- Is XNOR satisfies associative law?? – even function??
 - Slide 52 (Few important points)
- Quine McClusky method/ tabular method
 - Takes notes shortly from book
 - Draw the figures clearly



Introduction

- **Gate level minimization**
 - A design task to get **optimized** digital circuit
 - But difficult for **manual approach** when the number of **inputs** are **many**
 - Contemporary **computer applications** can easily minimize it efficiently
 - So, we need to understand the underlying mathematical description



Implementation of Boolean Function

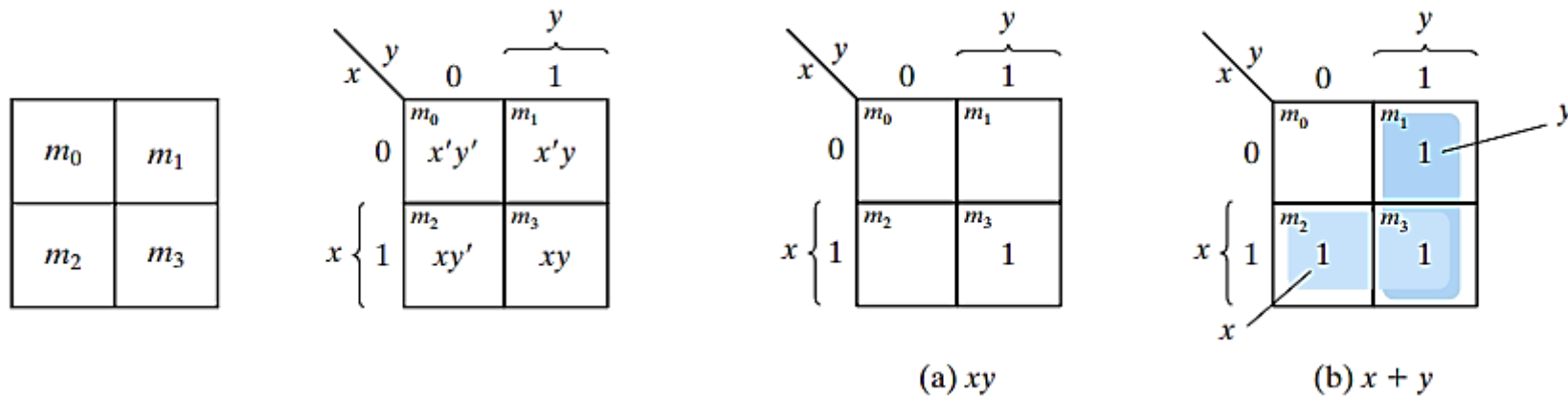
- **Complexity** of implementing digital logic gates of any Boolean function depends on the **complexity** of its algebraic expression.
 - A Boolean function can be expressed in **different ways**, but their **truth table** representation is **unique**
 - This Boolean expression can be simplified by Boolean algebra (**Cut-and-try method**)
 - But it's **manual** and there's **no specific rules** to proceed next step
- **Map method** or **Veitch/Karnaugh Map** or **K-map**
 - Simple and straightforward procedure to minimize Boolean function
 - Also be regarded as **pictorial form** of a **truth table**

The Map Method – K-map

- K-map is made of squares
 - Each square represents a minterm of the function
- Graphical representation of a Boolean function in the map **enclosing** those squares whose minterms are included in the function
 - This representation reveals all possible ways a function may be expressed in standard form (SOP or POS)
 - A designer should identify various patterns to derive **alternative simplified** algebraic expression
 - **A super optimized expression** contains **minimum number of terms** and with **smallest possible number of literals** in each term
 - But sometimes more than one expressions may satisfy this minimization criteria – all of them are correct

Two-Variable K-map

- Two variables – Four minterms
 - Four Squares in K-map
 - Another way to represent any of the 16 functions of two variables
 - Minterm corresponding to a square can be obtained from the concatenation of the row and column number



Note: A square is marked with 1, if that minterm asserts the function

Three-Variable K-Map

- Three variables – Eight minterms
 - Eight squares in k-map
- Minterms are expressed symbolically in binary equivalent sequence but arranged in k-map as **a sequence of reflected code**
 - Only one-bit changes in value from one adjacent column to the next
 - Any two adjacent squares in the map differ by only one variable
 - Basic property of K-map

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

		y				
		yz	00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$	
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'	
			z			

Three-Variable K-Map: Simplification

- If any two adjacent squares (vertically or horizontally) are considered as sum of them, those two minterms could be simplified to a single product term following the postulates of Boolean algebra (*i. e.* $y + y' = 1$).
 - Dissimilar variable will be removed
 - Example: m_5 and m_7
- Note:** Two squares are adjacent even though they don't touch each other
 - Like **world map**
 - Example: m_0 and m_2



m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6

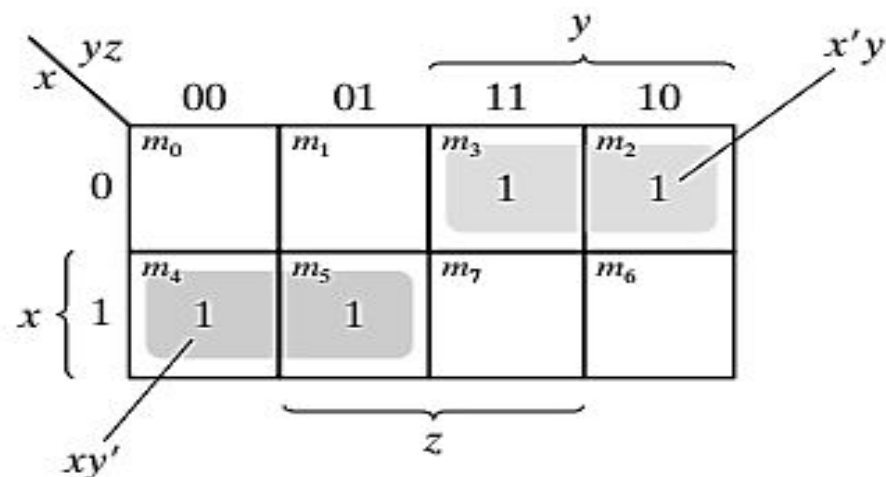
		y			
		00	01	11	10
x	0	m_0 $x'y'z'$	m_1 $x'y'z$	m_3 $x'yz$	m_2 $x'yz'$
	1	m_4 $xy'z'$	m_5 $xy'z$	m_7 xyz	m_6 xyz'
		z			

$$\begin{aligned}
 &m_5 + m_7 \\
 &= xy'z + xyz \\
 &= xz(y + y') \\
 &= xz
 \end{aligned}$$

3-variable K-Map: Example

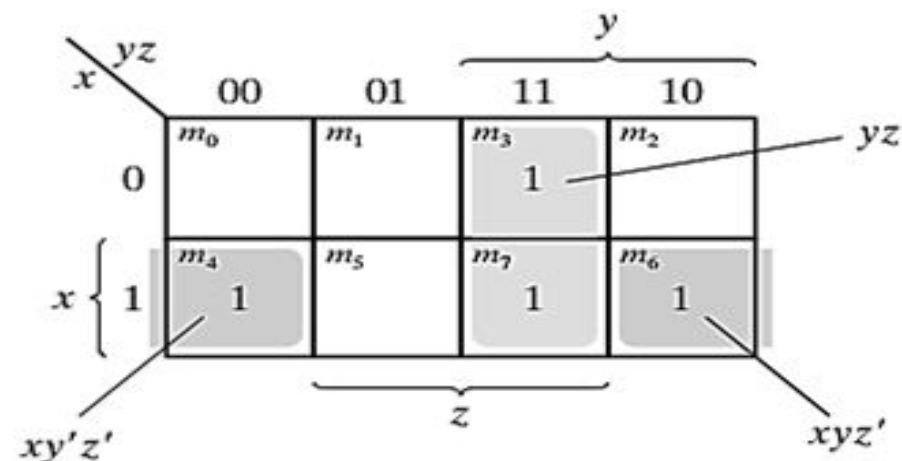
$$F(x, y, z) = \Sigma(2, 3, 4, 5)$$

$$F = x'yz + x'yz' + xy'z' + xy'z$$



$$F(x, y, z) = \Sigma(3, 4, 6, 7)$$

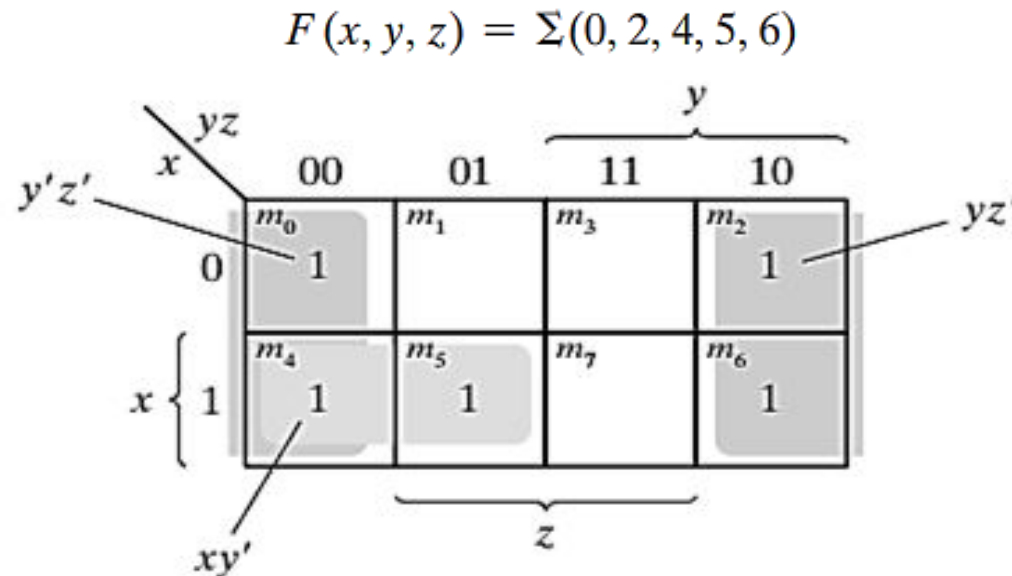
$$F = x'yz + xy'z' + xyz' + xyz$$



3-variable K-Map: Example...

Note:

- Like this, 4 adjacent squares are also possible following same postulates of Boolean algebra
- Use of squares more than once (**repeatedly**) makes a term more minimized
- **If the function is not expressed in canonical form, missing minterms should be obtained**
- Minimized function must be expressed in standard form



Solve these ?

$$F = A'C + A'B + AB'C + BC$$



4-Variable K-Map

- 4 variables – 16 minterms
 - 16 squares in K-map
- Both **columns** and **rows** are numbered in **Gray code** sequence
 - Only **one variable change** in any two adjacent squares row-wise or column-wise
- The map is considered on the surface whose **top and bottom edges** and **right and left edges** are attached to each other to form adjacent squares

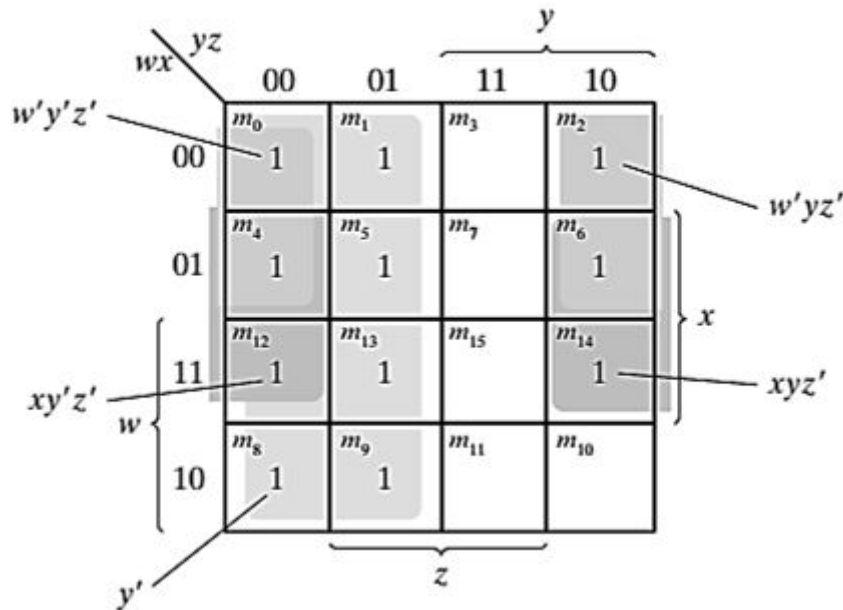
4-Variable K-Map: Representation

m_0	m_1	m_3	m_2
m_4	m_5	m_7	m_6
m_{12}	m_{13}	m_{15}	m_{14}
m_8	m_9	m_{11}	m_{10}

		y			
		yz	00	01	11
w	00	m_0 $w'x'y'z'$	m_1 $w'x'y'z$	m_3 $w'x'yz$	m_2 $w'x'yz'$
	01	m_4 $w'xy'z'$	m_5 $w'xy'z$	m_7 $w'xyz$	m_6 $w'xyz'$
	11	m_{12} $wxy'z'$	m_{13} $wxy'z$	m_{15} $wxyz$	m_{14} $wxyz'$
	10	m_8 $wx'y'z'$	m_9 $wx'y'z$	m_{11} $wx'yz$	m_{10} $wx'yz'$
		z			

4-Variable K-Map: Example

$$F(w, x, y, z) = \Sigma(0, 1, 2, 4, 5, 6, 8, 9, 12, 13, 14)$$



Solve this:

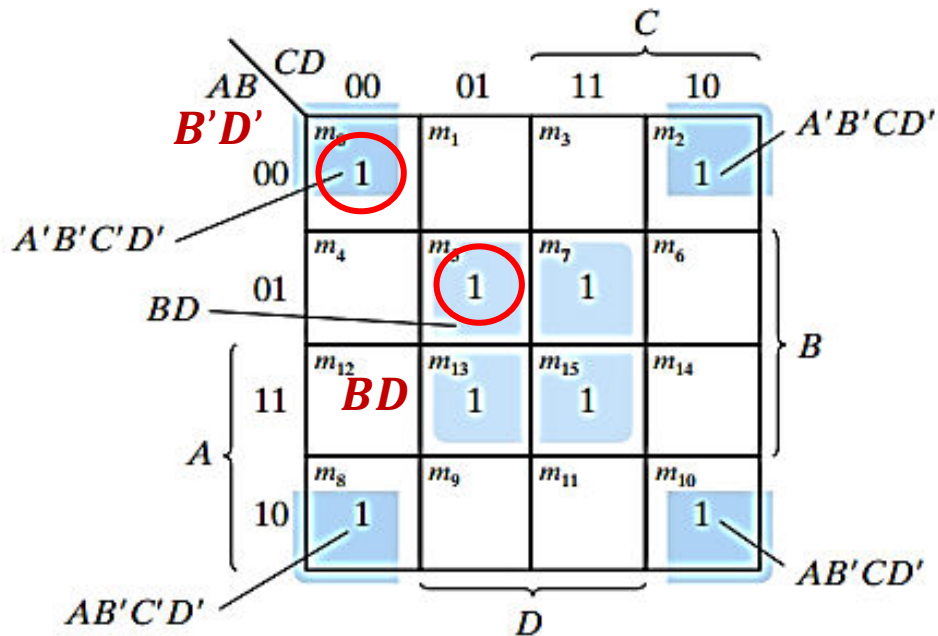
$$F = A'B'C' + B'CD' + A'BCD' + AB'C'$$

Basic Definition: Implicant

- **Implicant** is a **group** of **adjacent** squares in K-map consisting of 2^n **1s** representing true minterms in the function
 - Alternatively, **Implicant** is a product term (or minterm) of any function so that it implies the function (*i.e.*, if the implicant is true, the function will be true)
- A **prime implicant** is a product term obtained by combining the **maximum possible number of adjacent squares** in the map.
 - Where, this number satisfies 2^n
 - A single square itself represents a prime implicant if it has no other adjacent 1's square
- **Essential prime implicant**, is that prime implicant that has **at least one minterm** which is not covered by any other prime implicants.
 - This prime implicant **must** be included in the **simplified expression**

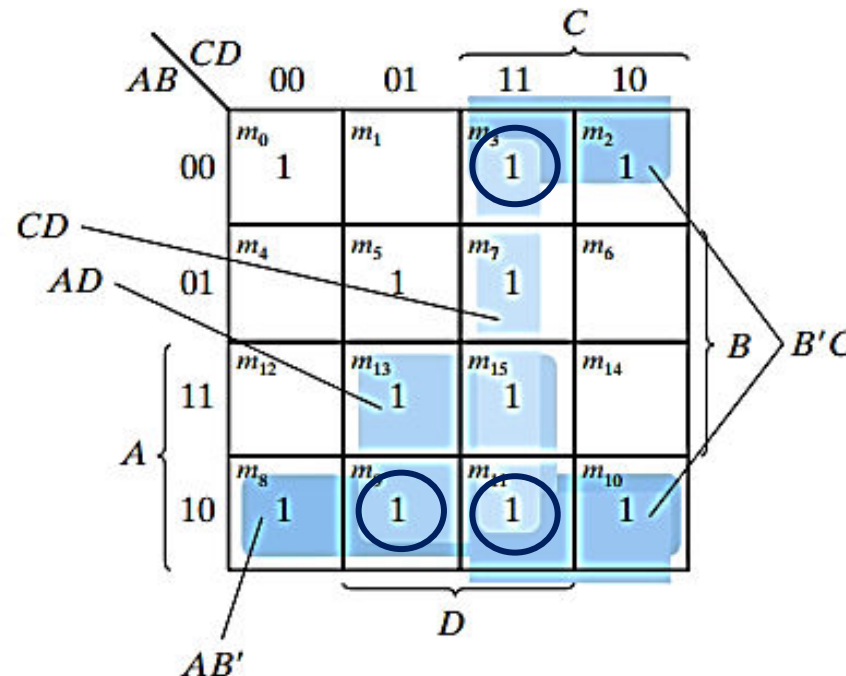
Basic Definition: Implicant – Example

- $F(A, B, C, D) = \sum(0, 2, 3, 5, 7, 8, 9, 10, 11, 13, 15)$



(a) Essential prime implicants
 BD and $B'D'$

1. The essential prime implicants are the only ways to cover some minterms



(b) Prime implicants $CD, B'C, AD,$ and AB'

2. There are more alternatives of prime implicants to cover remaining minterms

$$\begin{aligned}
 F &= BD + B'D' + CD + AD \\
 &= BD + B'D' + CD + AB' \\
 &= BD + B'D' + B'C + AD \\
 &= BD + B'D' + B'C + AB'
 \end{aligned}$$



Implicants in 3- and 4-variable K-map

- **3 variable K-map**

- **One** square – a term with three variables
 - One minterm itself
- **Two** adjacent squares – a term with two variables
- **Four** adjacent squares – a term with one variable
- **Eight** adjacent squares – always 1
 - Constant function

- **4 variable K-map**

- **One** square – a term with four variables
 - One minterm itself
- **Two** adjacent squares – a term with three variables
- **Four** adjacent squares – a term with two variables
- **Eight** adjacent squares – a term with one variable
- **Sixteen** adjacent squares – always 1
 - Constant function

5- and 6-Variable K-Map

CDE								
	000	001	011	010	110	111	101	100
AB								
00								
01								
11								
10								

5- variable Karnaugh map

		DEF				D				
		000	001	011	010	110	111	101	100	
ABC	000	0	1	3	2	6	7	5	4	C
	001	8	9	11	10	14	15	13	12	
	011	24	25	27	26	30	31	29	28	
	010	16	17	19	18	22	23	21	20	
A	110	48	49	51	50	54	55	53	52	B
	111	56	57	59	58	62	63	61	60	
	101	40	41	43	42	46	47	45	44	C
	100	32	33	35	34	38	39	37	36	
		F		E		F				

6-variable K-map

POS & SOP Simplification

- **SOP simplification** – so far, we have minimized the previous Boolean functions
 - With minor modification, **POS simplification** is also achievable
- **Procedure:**
 1. Mark the **empty squares** by **0's** in the K-map for any Boolean function
 2. Combine them into **valid adjacent squares**
 - A **simplified SOP expression** of the **complement of the function** (*i.e.*, F') is achieved
 3. The complement of F' gives back the function F in **simplified POS form** following the **De Morgan's law**

POS & SOP Simplification - Example

- $F(A, B, C, D) = \sum(0, 1, 2, 5, 8, 9, 10)$
 - **Simplification** in SOP and POS form

		C			
		00	01	11	10
A	00	m_0 1	m_1 1	m_3 0	m_2 1
	01	m_4 0	m_5 1	m_7 0	m_6 0
	11	m_{12} 0	m_{13} 0	m_{15} 0	m_{14} 0
	10	m_8 1	m_9 1	m_{11} 0	m_{10} 1
		D			

The Karnaugh map shows the function $F(A, B, C, D)$ with 1s at minterms 0, 1, 2, 5, 8, 9, and 10. The map is annotated with groupings for simplification:

- $BC'D'$ (Group 1: m0, m1)
- BCD' (Group 2: m4, m5)
- AB (Group 3: m8, m9)
- BCD (Group 4: m12, m13, m14, m15)

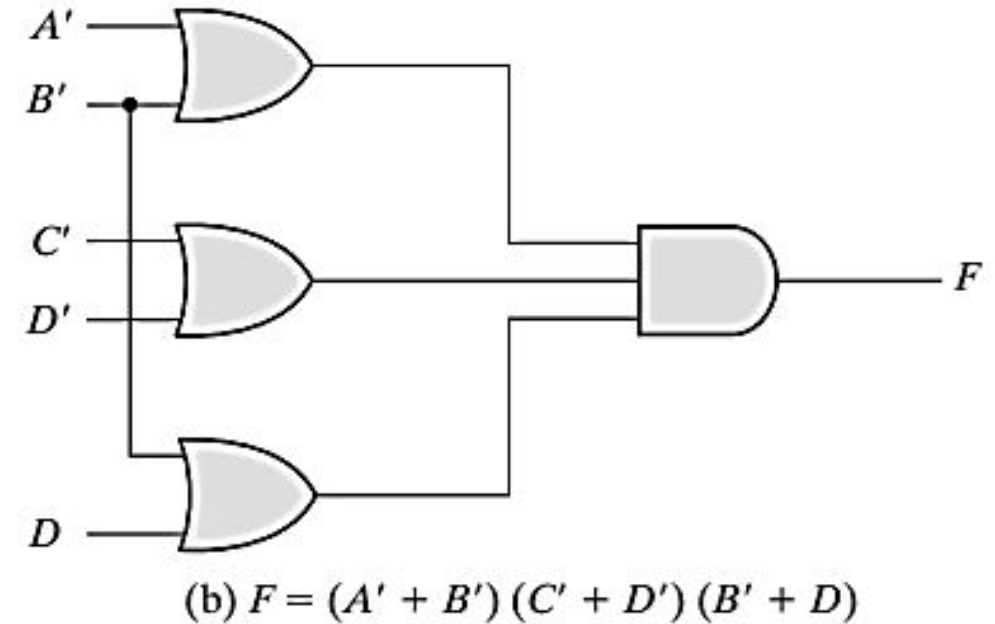
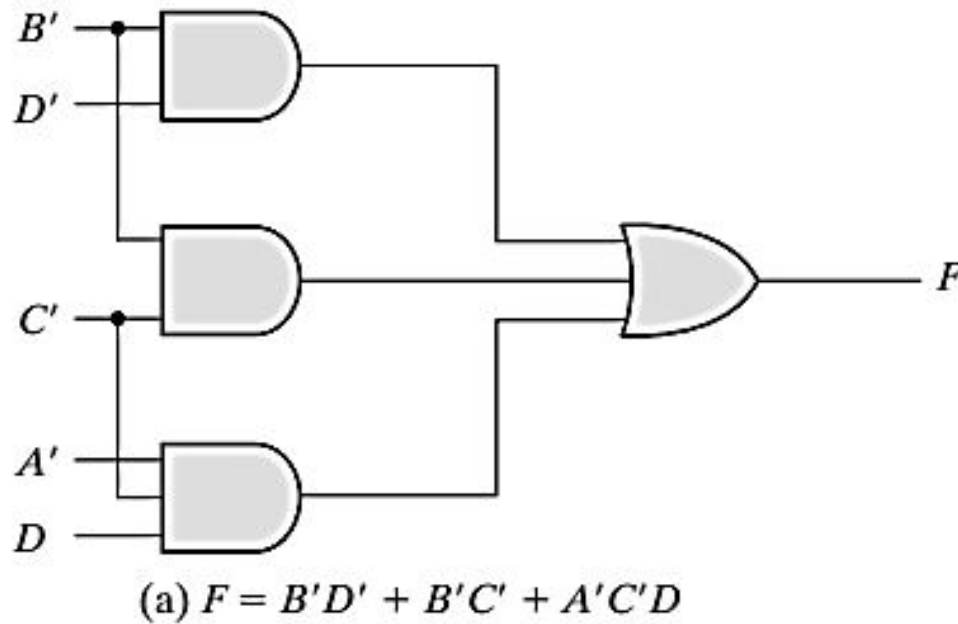
$$F(A, B, C, D) = \sum(0, 1, 2, 5, 8, 9, 10)$$

$$\text{SOP: } F = B'D' + B'C' + A'C'D$$

$$\text{POS: } F = (A' + B')(C' + D')(B' + D)$$

POS & SOP Simplification – Gate Level Implementation

- $F(A, B, C, D) = \sum(0,1,2,5,8,9,10)$
 - Gate level implementation of simplified SOP and POS expressions





POS & SOP Simplification – Important

- If the function is expressed in **canonical form using maxterms (POS)**, the simplified expression is also derivable if we consider those **minterms** in K-map marking with **0's**
 - **Then** we can account those **remaining squares** to get the simplified SOP expression of **F**
 - **Otherwise**, we can consider the **currently marked squares** with **0s** to get the simplified SOP expression of **F'**
- If any expression is expressed in **standard POS format**, we have to use the complement of that function to get its correspondent expression of **F'** in **standard SOP format**
 - **Example:** $F = (A' + B' + C')(B + D)$
 - **Then,** $F' = ABC + B'D'$ will be obtained to calculate all the minterms causes **0**.



Incompletely Specified Functions

Don't Care Conditions

- **Fact: Logical sum of minterms** (i.e., $\sum m_j$) of a Boolean function specifies the condition under which the function is **true (logic 1)**
 - The function is **false (logic 0)** for rest of the minterms
- This pair of conditions assumes that **all possible combinations** for the **variables** are valid for the function (either **logic 0** or **logic 1**)
 - **But in practice**, some combinations of variables may not be valid for a Boolean function in some applications
 - **Example:** BCD – a 4-bit binary code for decimal digits
 - There are six such combinations which are not used and considered as **unspecified**
- **Incompletely Specified Functions**
 - **Functions those have unspecified outputs for some input combinations**
 - We simply **don't care** what value will be produced by the function for those unspecified minterms
 - Those **unspecified minterms** of any function are called **don't care conditions**
 - **Useful for further simplification of any Boolean expression**



Don't Care Conditions

- A **don't care minterm** is a combination of variables whose logical value (0 or 1) is not specified (*i.e.*, **defined**) in the Boolean function
 - So that corresponding square can't be marked with **1** or **0** in the K-map
 - Rather "**X**" could be placed in that corresponding square distinguishing from 0 and 1
 - It indicates that we don't care whether the value of 0 or 1 is assigned to that function for that minterm
- **For better simplification**, adjacent don't care square may be assumed to be either 0 or 1
 - **Preference** of 0 or 1 depends on which combination gives the **simplest expression**
- **Trick:** To get the simplified expression of any Boolean function, we **must include** all 1s, but we may or may not include any of the Xs based on preference
 - Initially, those don't care minterms could be unspecified, but when a particular choice is made, they should be treated differently.

Don't Care Conditions – Example

$$F(w, x, y, z) = \Sigma(1, 3, 7, 11, 15)$$

$$d(w, x, y, z) = \Sigma(0, 2, 5)$$

		y			
		00	01	11	10
wx	yz				
w'x'	00	X	1	1	X
	01	0	X	1	0
wx	11	0	0	1	0
	10	0	0	1	0

(a) $F = yz + w'x'$

		y			
		00	01	11	10
wx	yz				
w'z	00	X	1	1	X
	01	0	X	1	0
wx	11	0	0	1	0
	10	0	0	1	0

(b) $F = yz + w'z$

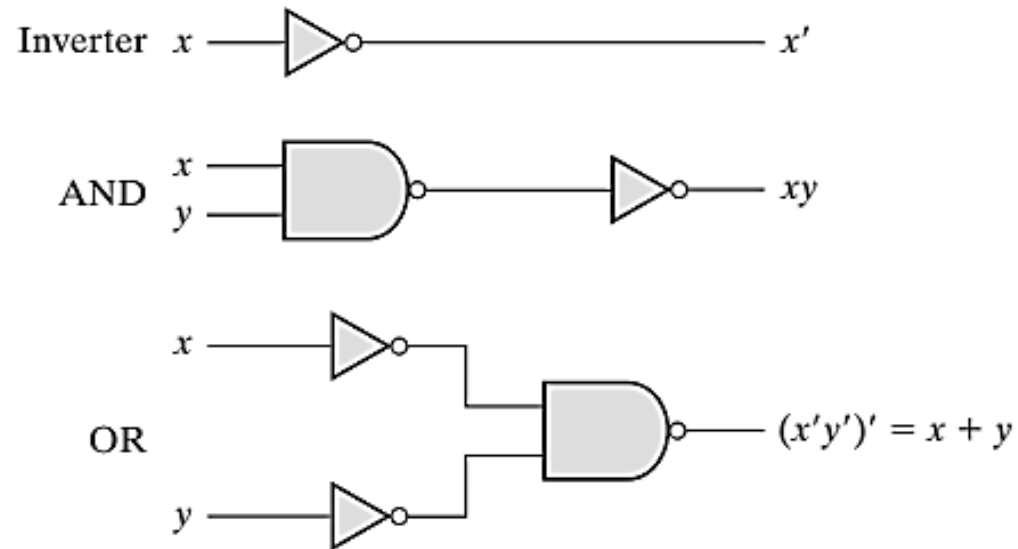
Note: It is also possible to simplify in POS form using its complement. *i.e.*, $F' = z' + wy'$

NAND & NOR implementation

- Digital circuits are **frequently constructed** with **NAND or NOR** gates rather than AND, OR and NOT gate **because**:
 - These gates are **easier to fabricate** with electronic components
 - They are used as **basic gates** in all ICs
 - **Any logic circuits** can be implemented with these gates
 - So, they are called **universal gates**
- Because of **prominence** of these gates, some rules and procedures have been developed to convert any circuit using AND, OR and NOT gates into equivalent circuit using NAND or NOR gates

NAND implementation

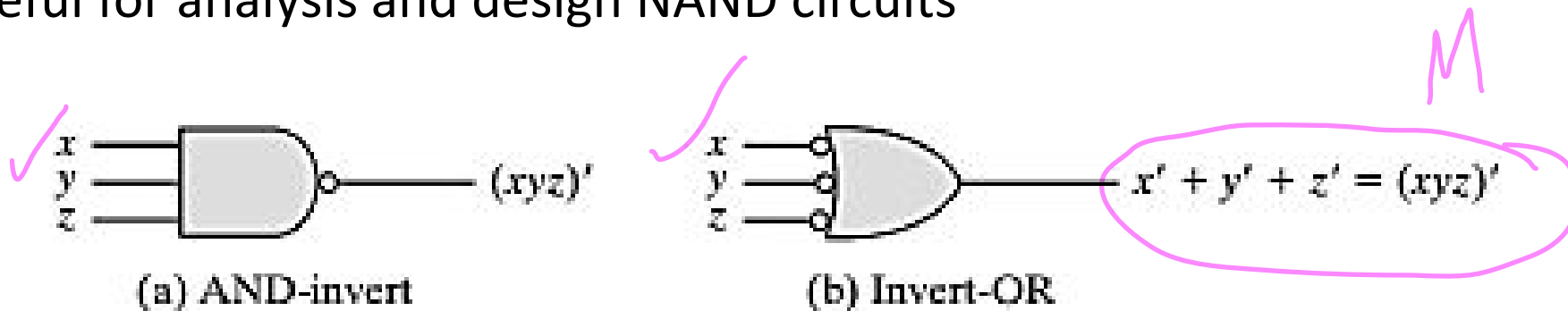
- To show that any function can be implemented with NAND gate, we need only to show AND, OR and NOT gates obtained with NAND gates alone



Note: One-input NAND gate behaves exactly like an inverter.

NAND Implementation: Graphic Symbols

- To facilitate the conversion to NAND logic, it is convenient to define an **alternative graphic symbol** for the NAND gate
 - AND-invert:** Consists of an AND gate followed by a small bubble (*i.e.*, complement)
 - Invert-OR:** Consists of an OR gate preceded by a bubble in each input (*i.e.*, complements each input)
 - Both of them are equal because of De Morgan's theorem
 - Useful for analysis and design NAND circuits



Two Level NAND Implementation

- For two level NAND implementation, **initially the function must be simplified in SOP form**
 - **AND gate** will be replaced by **AND-invert NAND gate** in the first level and **OR gate** will be replaced by **invert-OR NAND gate** in the second level
 - As Bubble denotes complementation, two cascaded bubbles will be cancelled by each other
 - But, if there is a **single-variable term** in first level of implementation, an inverter will be required for that term
 - We can replace that inverter changing the input variable into its complement (*i.e.*, x will be x' in the input)

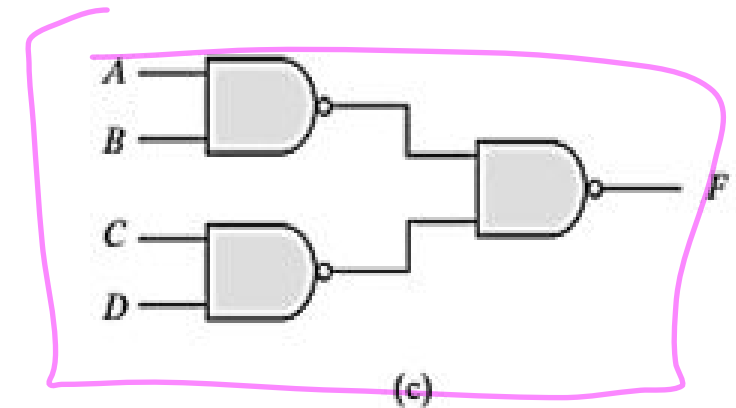
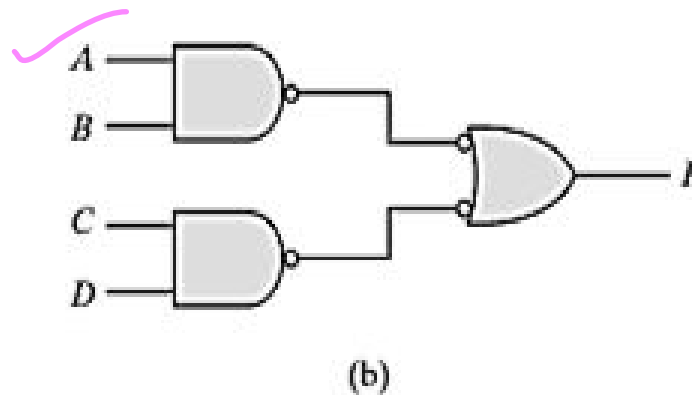
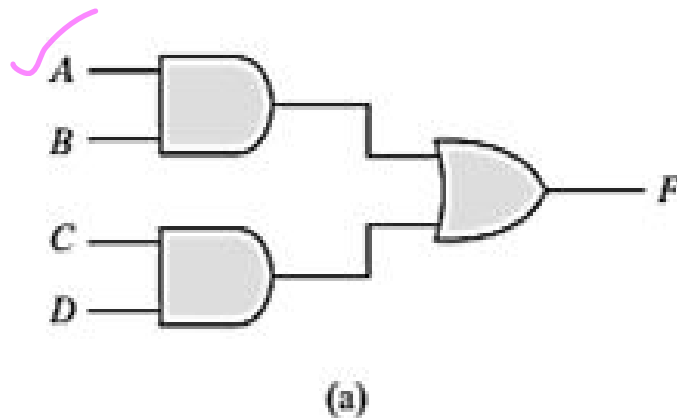


Two Level NAND Implementation: Procedure

1. Simplify the function and express it in **SOP form**
2. To implement the first level gates, all product terms of the expression (having at least two variables) drawn with NAND gates
 - Inputs of these NAND gates will be the same literals of the previous terms
3. Draw another single NAND gate (representing invert-OR) in the second level with inputs coming from outputs of the first level gates
4. A term having a single-variable term requires an inverter in the first level. However, if that variable is complemented, it can be connected directly to the input of the second level NAND gate

Two Level NAND Implementation - Example

- Example: $F = AB + CD$
 - The NAND implementation can be verified algebraically using De Morgan's law
 - Verification: $F = ((AB)'(CD)')' = AB + CD$



Solve this:

$$F(x, y, z) = (1, 2, 3, 4, 5, 7)$$

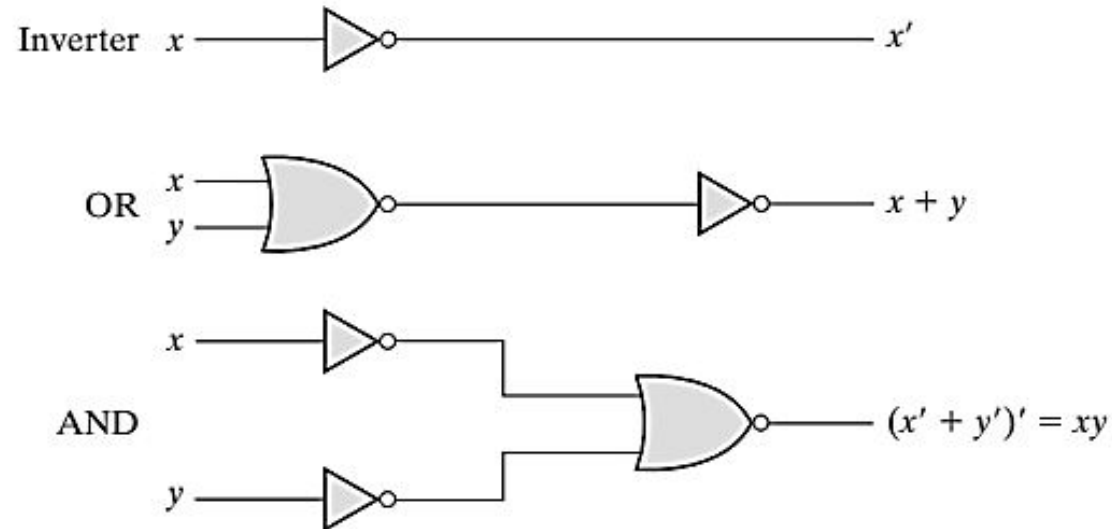


NOR implementation

- NOR operation is the dual of NAND operation
 - All procedures and rules for NOR logic are the dual of the corresponding procedures and rules developed for NAND logic
- Another ***universal gate*** to implement any logic circuit

NOR implementation

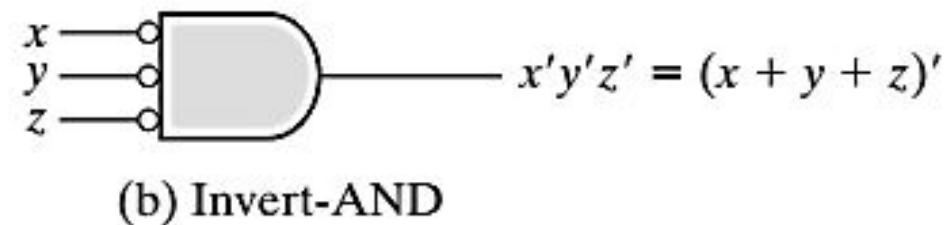
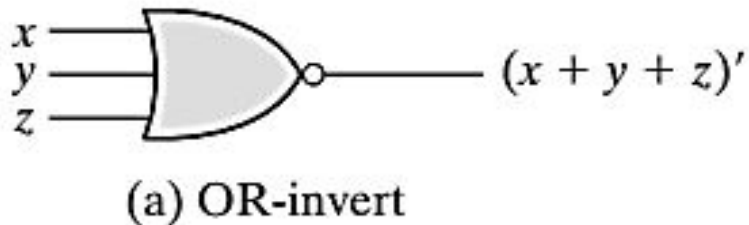
- To show that any function can be implemented with NOR gate, we need only to show that AND, OR and NOT gates obtained with NOR gates alone



Note: One-input NOR gate behaves exactly like an inverter.

NOR Implementation: Graphic Symbols

- To facilitate the conversion to NOR logic, it is convenient to define an **alternative graphic symbol** for the NOR gate
 - **OR-invert**: Consists of an OR gate followed by a small bubble (*i.e.*, complement)
 - **Invert-AND**: Consists of an AND gate preceded by a bubble in each input (*i.e.*, complements each input)
- Both of them are equal because of De Morgan's theorem
- Useful for analysis and design NOR circuits



Two Level NOR Implementation

- For two level NOR implementation, initially the function must be simplified in **POS form**
 - **Note:** Simplified POS expression can be obtained by combining all 0s and complementing that expression
 - **OR gate** will be replaced by **OR-invert NOR gate** in the first level and **AND gate** will be replaced by **invert-AND NOR gate** in second level
 - As Bubble denotes complementation, two cascaded bubbles will be cancelled by each other
 - But, if there is a single-variable term in first level of implementation, an inverter will be required for that term
 - We can replace that inverter changing the input variable into its complement (*i.e.*, x will be x' in the input)

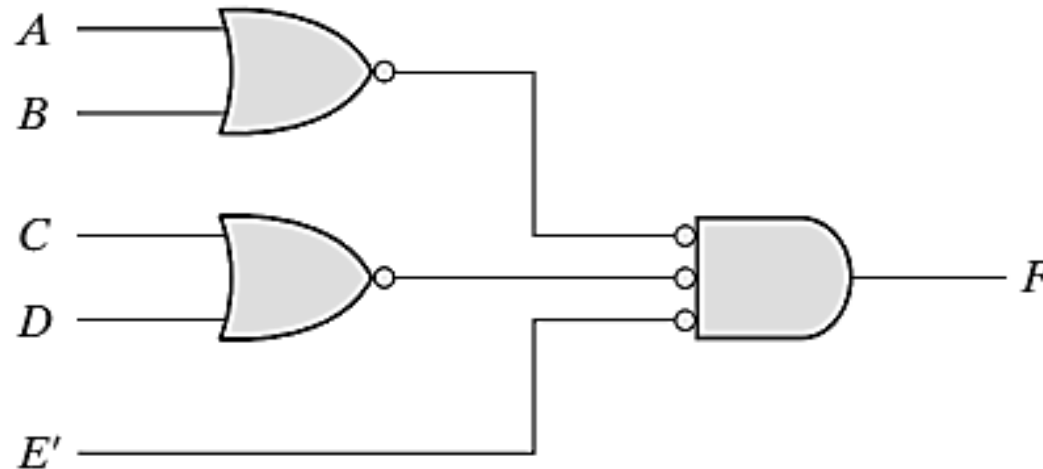


Two Level NOR Implementation: Procedure

1. Simplify the function and express it in **POS form**
2. To implement the first level gates, all sum terms of the expression (having at least two variables) drawn with NOR gates
 - Inputs of these NOR gates will be the same literals of the previous terms
3. Draw another single NOR gate (representing invert-AND) in the second level with inputs coming from outputs of the first level gates
4. A term having a single variable requires an inverter in the first level. However, if that variable is complemented, it can be connected directly to the input of the second level NOR gate

Two Level NOR Implementation - Example

- Example: $F = (A + B)(C + D)E$
 - The NOR implementation can be verified algebraically using De Morgan's law
 - Verification: $F = ((A + B)' + (C + D)' + E')' = (A + B)(C + D)E$



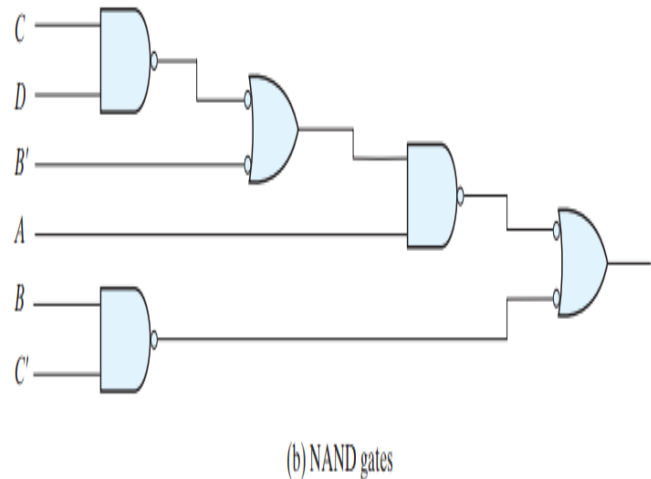
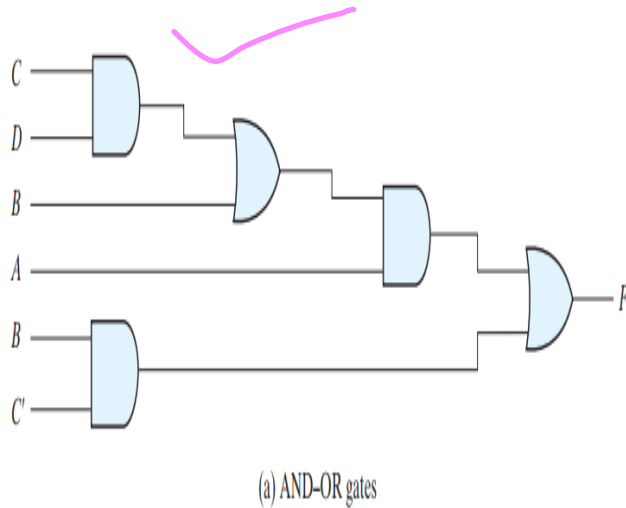


Multiple Level NAND Implementation

- **Standard form in SOP or POS** expression can be implemented in a **two-level** implementation
 - But sometimes, three or more level getting structure are also observed in digital systems
- There are **two approaches** to design a multi-level NAND circuits:
 1. Express the Boolean function in terms of AND, OR and NOT operators
 2. Two **alternative** ways:
 - **Approach 1:** All the AND, OR and NOT gates would be converted to its corresponding NAND representation
 - **Approach 2:** Convert each AND to AND-invert and each OR to Invert-OR gates
 3. There should have **two cascaded bubbles** or **inverters** who will cancel each other
 - Otherwise, one extra inverter should be inserted, or input variables should be complemented

Multiple Level NAND Implementation: Example

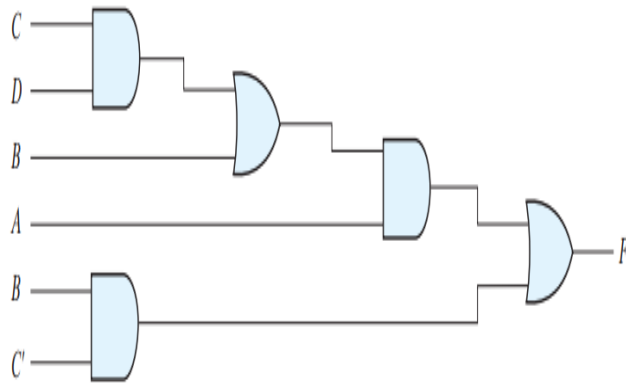
- **Example 1:** $F = A(CD + B) + BC'$ $= (A((CD)'B'))' \cdot (BC')'$
 - We could simplify this expression to its SOP form, but here a multilevel NAND circuit is preferred



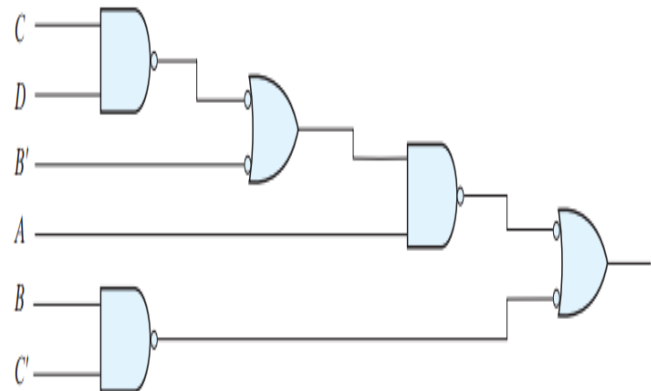
Approach 1

Multiple Level NAND Implementation: Example

- Example 1: $F = A(CD + B) + BC'$
 - We could simplify this expression to its SOP form, but here a multilevel NAND circuit is preferred



(a) AND-OR gates

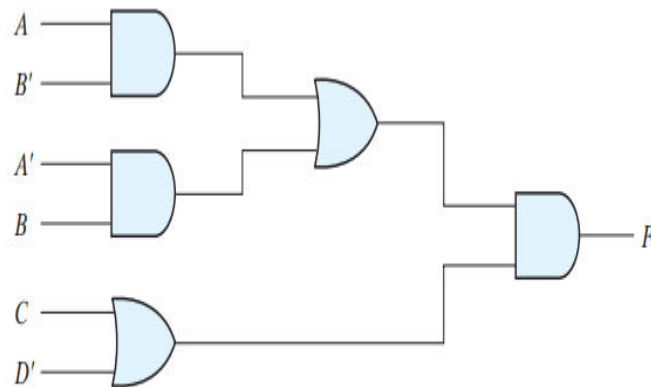


(b) NAND gates

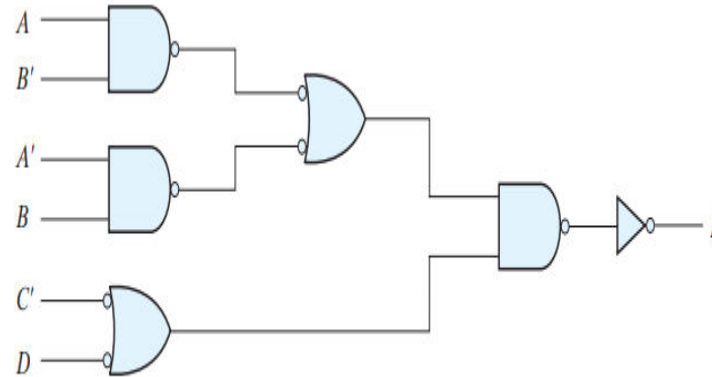
Approach 2

Multiple Level NAND Implementation: Example...

- Example 2: $F = (AB' + A'B)(C + D')$
 - We could simplify this expression to its SOP form, but here a multilevel NAND circuit is preferred



(a) AND-OR gates

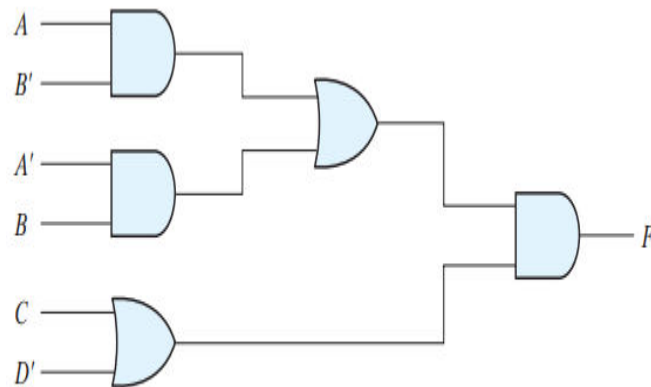


(b) NAND gates

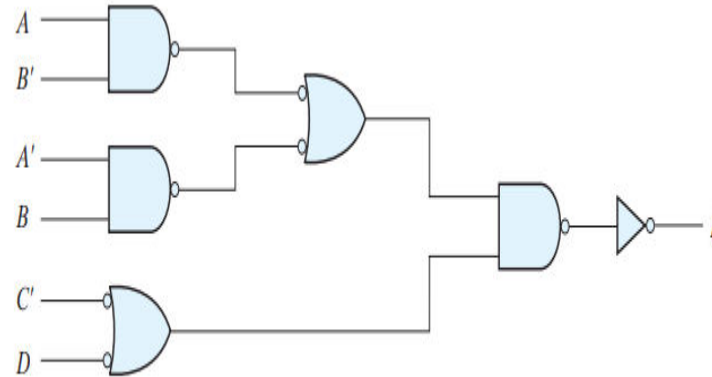
Approach 1

Multiple Level NAND Implementation: Example...

- Example 2: $F = (AB' + A'B)(C + D')$
 - We could simplify this expression to its SOP form, but here a multilevel NAND circuit is preferred



(a) AND-OR gates



(b) NAND gates

Approach 2

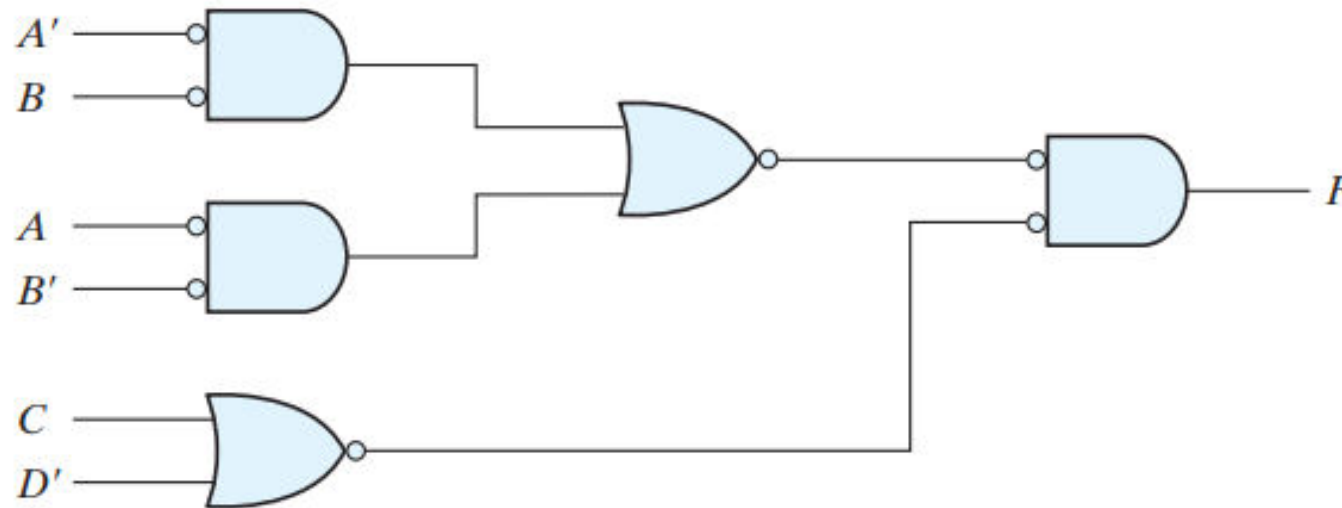


Multiple Level NOR Implementation

- There are two approaches to design a multi level NOR circuits:
 1. Express the Boolean function in terms of AND, OR and NOT operators
 2. Two **alternative** ways:
 - **Approach 1:** All the AND, OR and NOT gates would be converted to its corresponding NOR representation
 - **Approach 2:** Convert each AND to invert-AND and each OR to OR-invert gates
 3. There should have **two cascaded bubbles or inverters** who will cancel each other
 - Otherwise, one extra inverter should be inserted, or input variables should be complemented

Multiple Level NOR Implementation- Example

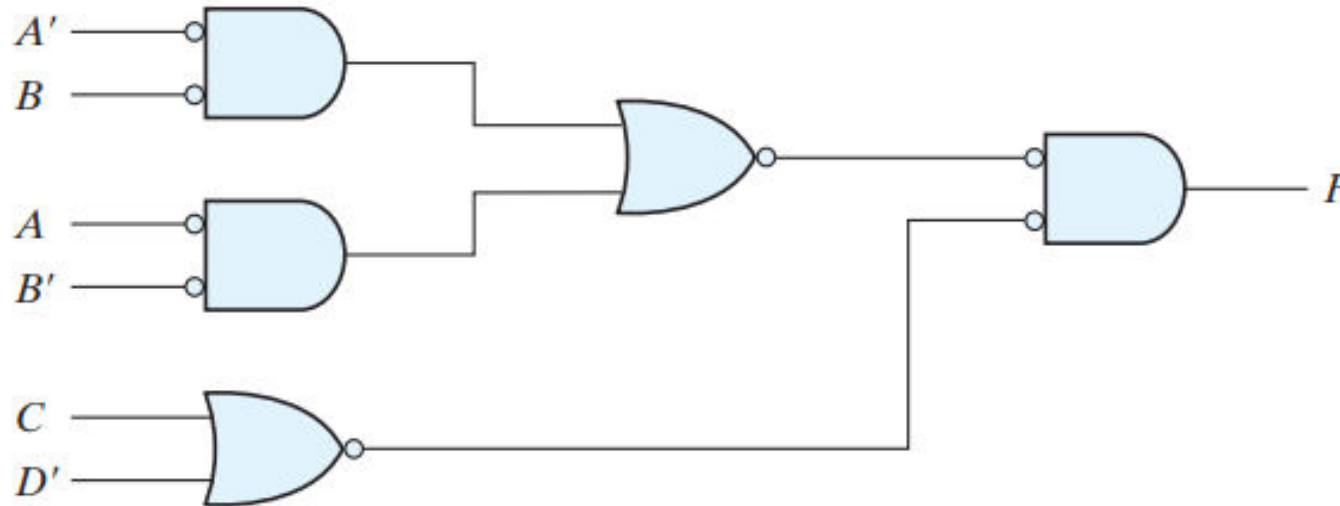
- Example 1: $F = (AB' + A'B)(C + D')$
 - We could simplify this expression to its POS/SOP form, but here a multilevel NOR circuit is preferred



Approach 1

Multiple Level NOR Implementation- Example...

- Example 1: $F = (AB' + A'B)(C + D')$
 - We could simplify this expression to its POS/SOP form, but here a multilevel NOR circuit is preferred



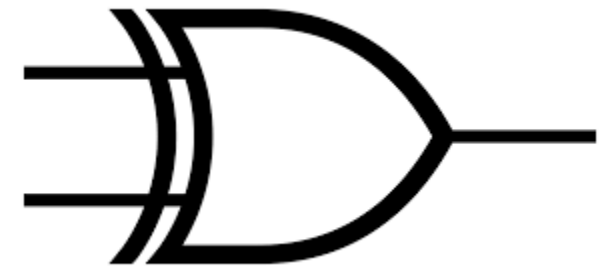
Approach 2

Exclusive-OR Function

- Exclusive-OR (XOR) symbolized as $\oplus$.
 - Example: $x \oplus y$
- It performs the Boolean operation: $x \oplus y = x'y + xy'$
 - The XOR function is 1 if only x is 1 or only y is 1 – x and y are differed in value
 - So, it is called **difference function**
 - For multiple variables, XOR is 1 if the total number of 1s in the input is odd.
- For XOR operation, following statements are true:
 - $x \oplus 0 = x$
 - $x \oplus 1 = x'$
 - $x \oplus x = 0$
 - $x \oplus x' = 1$
 - $x \oplus y' = x' \oplus y = (x \oplus y)'$

A	B	Q
0	0	0
0	1	1
1	0	1
1	1	0

XOR



Exclusive-NOR Function

- Exclusive-NOR (XNOR) symbolized as $\odot$

- Example: $x \odot y$

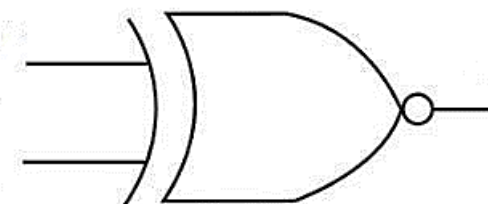
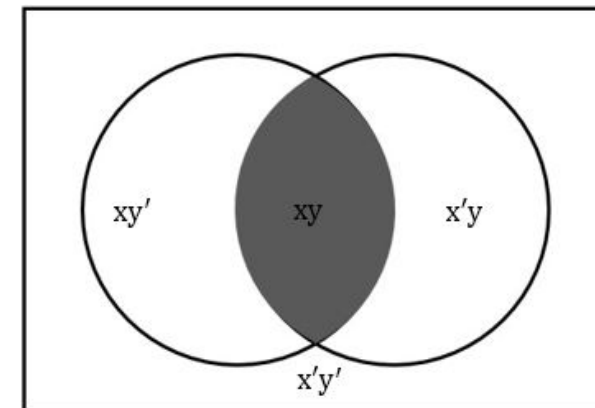
- It performs the Boolean operation: $x \odot y = xy + x'y'$

- By definition, XNOR function is the complement of the XOR
i.e.

$$x \odot y = (x \oplus y)' = (x'y + xy')' = (x + y')(x' + y) = xy + x'y'$$

- The XNOR function is 1 when both x and y are 1 or both of them are 0

- So, it is known as **equivalence function** considering two variables gate
 - For multiple input, XNOR is 1 if the total number of 1s in the input is even



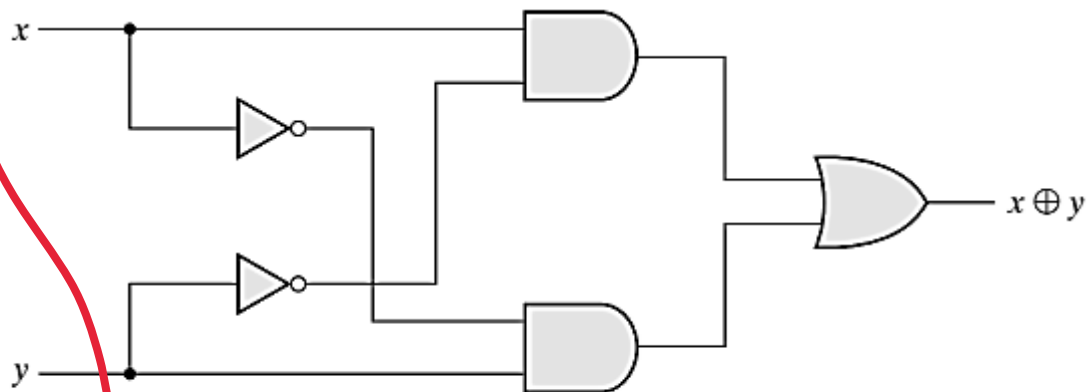
A	B	Q
0	0	1
0	1	0
1	0	0
1	1	1

XNOR

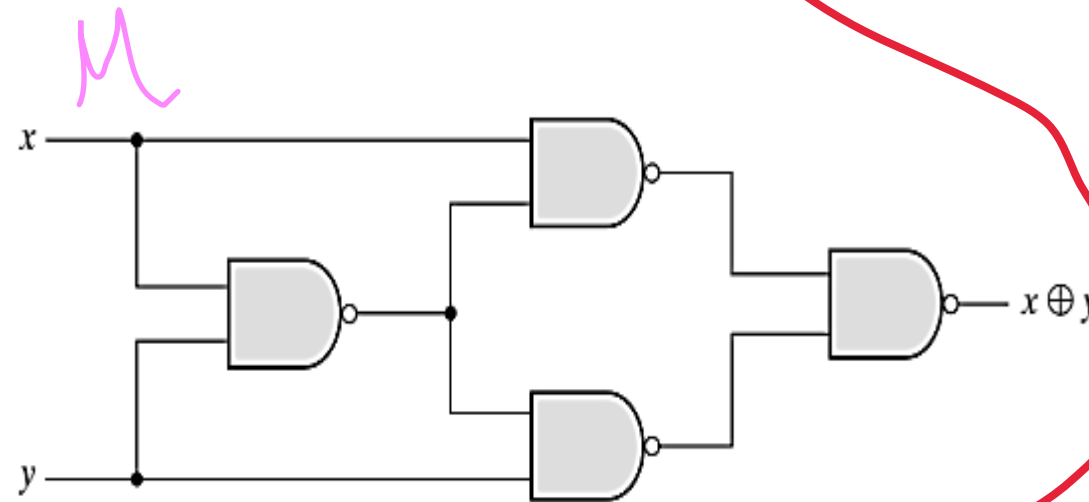
XOR an XNOR Function...

- Exclusive-OR (XOR) operation is both commutative and associative:
 - $A \oplus B = B \oplus A$
 - $(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$
- Exclusive-NOR (XNOR) operation is also both commutative and associative:
 - $A \odot B = B \odot A$
 - $(A \odot B) \odot C = A \odot (B \odot C) = A \odot B \odot C$
- So, the two inputs of XOR (or XNOR) gate can be interchanged without affecting the operation
- Also, we can evaluate a three or more inputs XOR (or XNOR) gate in any order
 - No need to include parentheses for further clarification
 - But multiple input XOR and XNOR gates are very difficult to fabricate with hardware

Implementation of XOR Gate



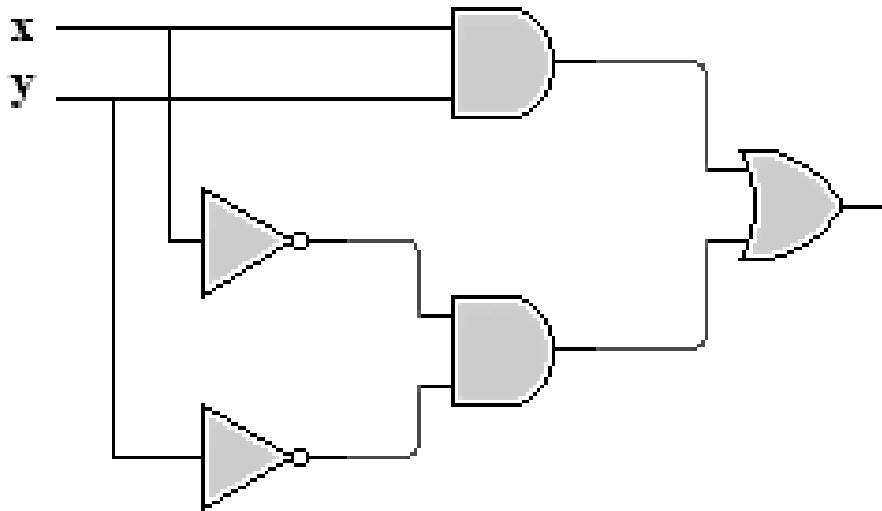
(a) Exclusive-OR with AND-OR-NOT gates



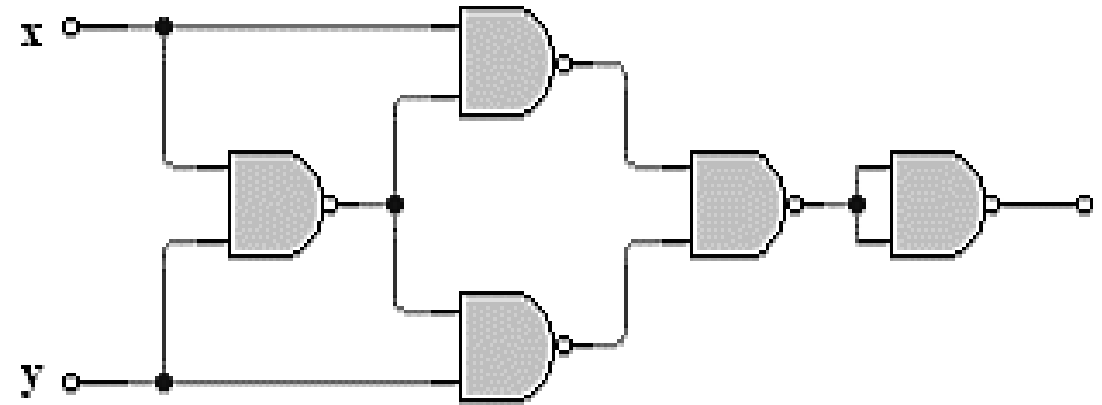
(b) Exclusive-OR with NAND gates

$$x'y + xy'$$

Implementation of XNOR Gate



XNOR gate with AND-OR-NOT gates



XNOR gate with NAND gates



Usage of XOR and XNOR Gates

- Only a limited number of Boolean functions can be expressed with XOR and XNOR gates
- In some digital system designs, they could be used
 - Specially, they are useful in -
 - Arithmetic operations (**e.g.**, add, subtract, etc.)
 - Logical operation (**e.g.**, complement)
 - Error detection and correction (**e.g.**, parity generator and checker)
 - Gray code conversion
- XOR gate is used extensively, XNOR could be implemented as XOR-inverter.

Odd Function with XOR gate

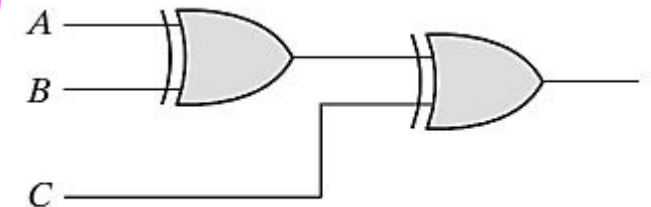
Considering Logic-1

- Basically, two input XOR gate could be defined as difference function
 - But, if we increase the number of input to three or more, definition will be changed

- Example:

$$\begin{aligned}
 A \oplus B \oplus C &= (AB' + A'B)C' + (AB + A'B')C \\
 &= AB'C' + A'BC' + ABC + A'B'C \\
 &= \Sigma(1, 2, 4, 7)
 \end{aligned}$$

A	B	C	Q = A⊕B⊕C
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



(a) 3-input odd function

Odd number of 1's make 1

- Here, three input XOR gate produces 1 if only one variable is 1 or all three variables are 1
- Definition:** A multiple input XOR gate operates as an **odd function i.e.** an odd number of inputs should be 1.
 - An n-variable XOR function is an odd function defined as the logical sum of the $2^n/2$ minterms whose binary numerical values have an odd number of 1s



Odd and Even Function with Four Variables

Considering Logic-1

- Algebraic expression of odd function with 4 variables:

$$\begin{aligned}A \oplus B \oplus C \oplus D &= (AB' + A'B) \oplus (CD' + C'D) \\&= (AB' + A'B)(CD + C'D') + (AB + A'B')(CD' + C'D) \\&= \Sigma(1, 2, 4, 7, 8, 11, 13, 14)\end{aligned}$$

- Algebraic expression of even function with 4 variables:

$$(A \oplus B \oplus C \oplus D)' = \Sigma(0, 3, 5, 6, 9, 10, 12, 15)$$

Input A	Input B	Input C	Input D	Output Y
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



*Few Important Points****

- For same odd number of variables, the XOR and XNOR function will produce same truth table
- For same even number of variables, the XOR and XNOR function will produce opposite (complementary) truth table

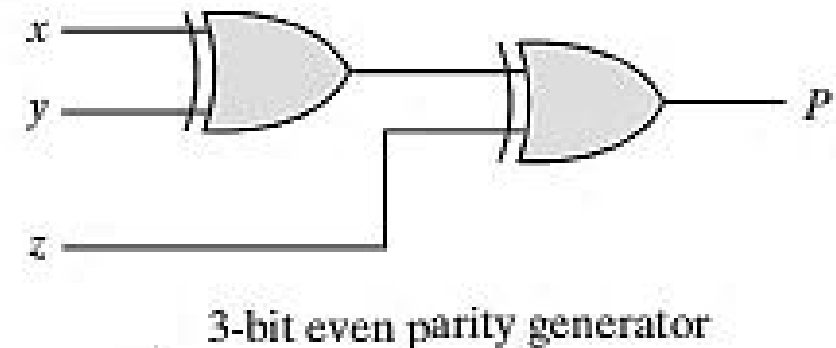
Application: Parity Generator and Checker

- XOR function is very useful for error detection and correction codes
- **Parity bit** is an extra bit which is usually included with original message during transmission for error detection
 - **Parity Generator** – the circuit that generates the parity bit in the transmitter
 - **Parity Checker** – the circuit that checks the parity in the receiver
- Example:
 - Consider a three bit message (*e.g.* x, y, z) to be transmitted together with an **even parity bit** (*e.g.* P)

Parity Generator Circuit

- In this circuit, three bits (*i.e.* x, y, z) in the message are inputs and even parity bit (*i.e.* P) is the output
 - For even parity, the bit P must be produced to make the total number of 1s (including P) even.
 - So, P makes an odd function, *i.e.* $P = x \oplus y \oplus z$

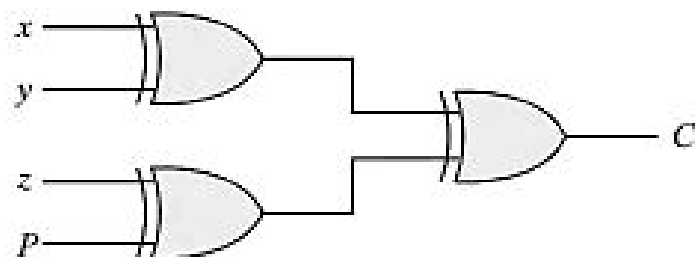
Three-Bit Message			Parity Bit
x	y	z	P
0	0	0	0
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1



Parity Checker Circuit

even parity er khetre 0 ashbe jodi 1 ashe tahole error easily detect kora jabe

- In this circuit, four bits (*i.e.* x , y , z and P) in the transmitted message are inputs and error checker bit (*i.e.* C) is the output
- For even parity checking, the checker bit C is 1 if an error occurs, *i.e.* if the total number of 1s in the received message is odd.
- So, C makes another odd function, *i.e.* $C = x \oplus y \oplus z \oplus P$



4-bit even parity checker

Four Bits Received				Parity Error Check
x	y	z	P	C
0	0	0	0	0
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	0



Prime Implicants Selections



Prime Implicants Selections

The Tabular Method / Quine-McCluskey Method

$$F = \Sigma(0, 1, 2, 8, 10, 11, 14, 15)$$

(a)						(b)						(c)					
$w x y z$						$w x y z$						$w x y z$					
0	0	0	0	0	✓	0, 1	0	0	0	–		0, 2, 8, 10	–	0	–	0	
						0, 2	0	0	–	0	✓	0, 8, 2, 10	–	0	–	0	
1	0	0	0	1	✓	0, 8	–	0	0	0	✓	10, 11, 14, 15	1	–	1	–	
2	0	0	1	0	✓							10, 14, 11, 15	1	–	1	–	
8	1	0	0	0	✓	2, 10	–	0	1	0	✓						
						8, 10	1	0	–	0	✓						
10	1	0	1	0	✓	10, 11	1	0	1	–	✓						
11	1	0	1	1	✓	10, 14	1	–	1	–	✓						
14	1	1	1	0	✓												
15	1	1	1	1	✓	11, 15	1	–	1	1	✓						
						14, 15	1	1	1	–	✓						

$$F = w'x'y' + x'z' + wy$$

The Tabular Method (Alternative Way)

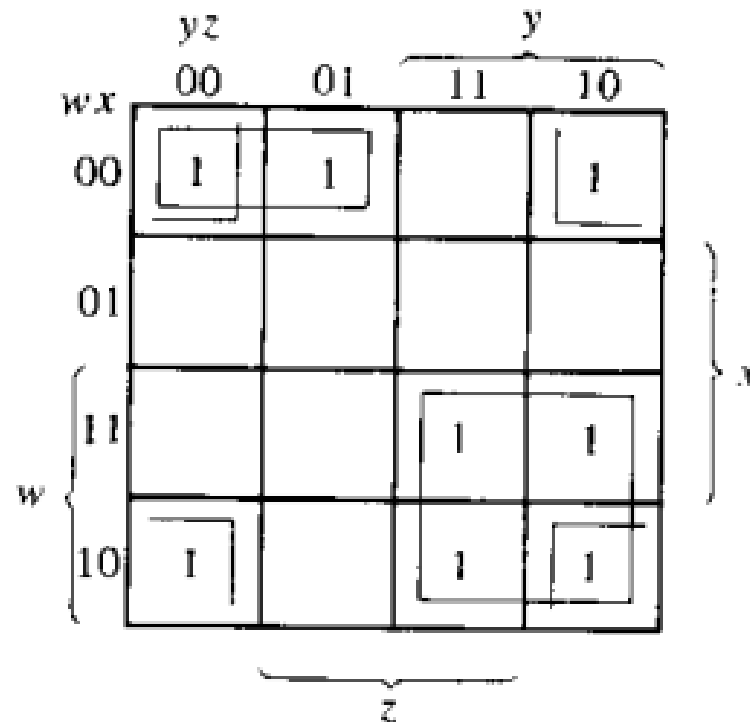
TABLE 3-6
Determination of Prime Implicants of Example 3-13 with Decimal Notation

(a)	(b)	(c)
0 ✓	0, 1 (1)	0, 2, 8, 10 (2, 8)
	0, 2 (2) ✓	0, 2, 8, 10 (2, 8)
1 ✓	0, 8 (8) ✓	
2 ✓		10, 11, 14, 15 (1, 4)
8 ✓	2, 10 (8) ✓	10, 11, 14, 15 (1, 4)
	8, 10 (2) ✓	
10 ✓		
11 ✓	10, 11 (1) ✓	
	10, 14 (4) ✓	
14 ✓		
	11, 15 (4) ✓	
15 ✓	14, 15 (1) ✓	

(difference)
(old diff, new diff)

The Tabular Method

- Verification using K-map:



Prime Implicants Determination

- Another Example:

$$F(w, x, y, z) = \Sigma(1, 4, 6, 7, 8, 9, 10, 11, 15)$$

Determination of Prime Implicants for Example 3-14

(a)			(b)		(c)
0001	1	✓	1, 9	(8)	8, 9, 10, 11 (1, 2)
0100	4	✓	4, 6	(2)	8, 9, 10, 11 (1, 2)
1000	8	✓	8, 9	(1) ✓	
			8, 10	(2) ✓	
0110	6	✓			
1001	9	✓	6, 7	(1)	
1010	10	✓	9, 11	(2) ✓	
			10, 11	(1) ✓	
0111	7	✓			
1011	11	✓	7, 15	(8)	
			11, 15	(4)	
1111	15	✓			

Prime Implicants Selections

Prime implicants					
Decimal	Binary				Term
	w	x	y	z	
1, 9 (8)	—	0	0	1	$x'y'z$
4, 6 (2)	0	1	—	0	$w'xz'$
6, 7 (1)	0	1	1	—	$w'xy$
7, 15 (8)	—	1	1	1	xyz
11, 15 (4)	1	—	1	1	wyz
8, 9, 10, 11 (1, 2)	1	0	—	—	wx'

Prime Implicant Table for Example 3-15

		1	4	6	7	8	9	10	11	15
✓ $x'y'z$	1, 9	X					X			
✓ $w'xz'$	4, 6		X	X						
$w'xy$	6, 7			X	X					
xyz	7, 15				X					X
wyz	11, 15								X	X
✓ wx'	8, 9, 10, 11					X	X	X	X	
		✓	✓	✓		✓	✓	✓	✓	

The Tabular Method

Verification using K-map:

		yz					
		00	01	11	10		
wx	00		1				
	01	1		1	1		
	11			1			
	10	1	1	1	1		

$\underbrace{\hspace{10em}}_y$
 $\underbrace{\hspace{10em}}_x$

$$F = x'y'z + w'xz' + wx' + xyz$$







Administration



Administration



Administration



Administration



Administration



Administration



Administration